MongoDB

Time Series in MongoDB

Time Series Collections, Aggregation Pipeline and Data Lifecycle Management



Jackson Nguyen

Solutions Architect, MongoDB | October 2026

AGENDA

1. What Is Time Series Data?
2. Time Series Collections
3. Aggregation Pipeline
4. Data Lifecycle Management
5. Hackathon Cheat Sheet
6. Q&A

01

WHAT IS TIME SERIES DATA?

Time Series Data Is Everywhere

Source	What gets measured	How often
Smartwatch	Heart rate, steps	Every second
Delivery app	Driver's GPS location	Every 5 to 10 seconds
Stock market	Price of VNM, FPT, VCB	On every trade
Air quality sensor	PM2.5, temperature, humidity	Every minute
Server monitoring	CPU, memory, requests per second	Every 10 to 60 seconds

One Reading, Three Parts

```
{
  "ts": ISODate("2026-10-13T08:00:00Z"),
  "sensor": {
    "id": "HCM-Q1-007",
    "city": "HCMC"
  },
  "pm25": 38.2,
  "temp": 31.5,
  "humidity": 74
}
```

timeField

When it was measured. A real Date, not a string.

metaField

Which sensor, where. Rarely changes.

measurements

The values. Different every reading.

All readings with the same metaField value form one series, for example everything sensor HCM-Q1-007 has ever sent.

Why Not Just Use a Normal Collection?

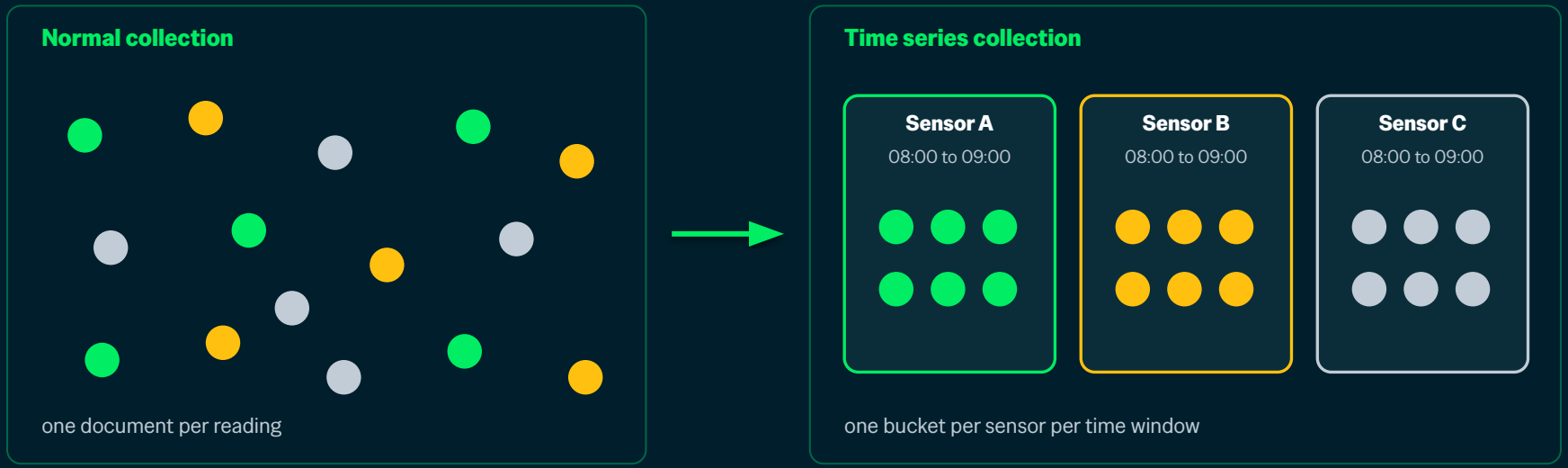
1,440	× 10,000	× 365	= 5.2B
readings per sensor per day	sensors	days	documents in a year

In a normal collection every reading is its own document, so the sensor ID and city are stored billions of times, the indexes grow as fast as the data, and "average PM2.5 last week" has to read millions of documents.

02

TIME SERIES COLLECTIONS

How It Works: Buckets



You still insert and query single readings. MongoDB does the grouping. A bucket holds up to 1,000 readings or 125 KB.

What a Bucket Looks Like

YOU INSERT 3 DOCUMENTS

```
{ ts: 08:00,
  sensor: { id: "Q1-007" },
  pm25: 35.1 }

{ ts: 08:01,
  sensor: { id: "Q1-007" },
  pm25: 36.4 }

{ ts: 08:02,
  sensor: { id: "Q1-007" },
  pm25: 38.2 }
```



MONGODB STORES 1 BUCKET

```
{
  meta: { id: "Q1-007" },     // stored once

  control: {                  // lets MongoDB
    min: { ts: 08:00, pm25: 35.1 },
    max: { ts: 08:02, pm25: 38.2 }
  },                          // skip buckets

  data: {                     // one column
    ts:   [ 08:00, 08:01, 08:02 ],
    pm25: [ 35.1,  36.4,  38.2  ]
  }                           // per field
}
```

Create a Time Series Collection

```
db.createCollection("air_quality", {
  timeseries: {
    timeField:   "ts",
    metaField:   "sensor",
    granularity: "minutes"
  },
  expireAfterSeconds: 7776000 // 90 days
})

db.air_quality.insertOne({
  ts: new Date(),
  sensor: { id: "HCM-Q1-007", city: "HCMC" },
  pm25: 38.2,
  temp: 31.5
})
```

`timeField`

Required. The field holding the date. Cannot be changed later.

`metaField`

Strongly recommended. Names the source. Cannot be changed later.

`granularity`

"seconds", "minutes" or "hours". Can be raised later, never lowered.

`expireAfterSeconds`

Optional. Deletes readings older than this.

Choosing the metaField

GOOD

```
{ ts: ...,
  sensor: { id: "HCM-Q1-007", city: "HCMC" },
  pm25: 38.2 }

// metaField: "sensor"
```

10,000 sensors means about 10,000 open buckets. Each one fills up with that sensor's readings.

AVOID

```
{ ts: ...,
  info: { id: "HCM-Q1-007", requestId: "a81f..."
},
  pm25: 38.2 }

// metaField: "info"
```

requestId is different every time, so every reading gets its own bucket. No grouping, no compression.

Put in the metaField only labels you filter on and that rarely change: device ID, city, type. Never measured values, IDs per reading or arrays.

Choosing Granularity

granularity	One source sends data	A bucket covers up to	Example
`"seconds"`	every few seconds (default)	1 hour	GPS from delivery drivers
`"minutes"`	about once a minute	24 hours	Our air quality sensors
`"hours"`	once an hour or less	30 days	Hourly weather station summary

Look at one sensor, not the whole system: 10,000 sensors may send data every second in total, but each one only sends once a minute.

For exact control on MongoDB 7.0+, set bucketMaxSpanSeconds and bucketRoundingSeconds to the same value instead.

Inserting Data

```
db.air_quality.insertMany([
  { ts: ISODate("2026-10-13T08:00:00Z"),
    sensor: { id: "HCM-Q1-007" }, pm25: 35.1 },
  { ts: ISODate("2026-10-13T08:01:00Z"),
    sensor: { id: "HCM-Q1-007" }, pm25: 36.4 },
  { ts: ISODate("2026-10-13T08:00:00Z"),
    sensor: { id: "HN-HK-002" },  pm25: 52.0 },
  { ts: ISODate("2026-10-13T08:01:00Z"),
    sensor: { id: "HN-HK-002" },  pm25: 54.3 }
], { ordered: false })
```

1. **Use insertMany**

   Hundreds of readings per call, not one insertOne each.

2. **Keep a sensor's readings together**

   They end up in the same bucket.

3. **ordered: false**

   One bad document does not stop the batch.

4. **Oldest first, same field order**

   Out-of-order data still works, but compresses worse.

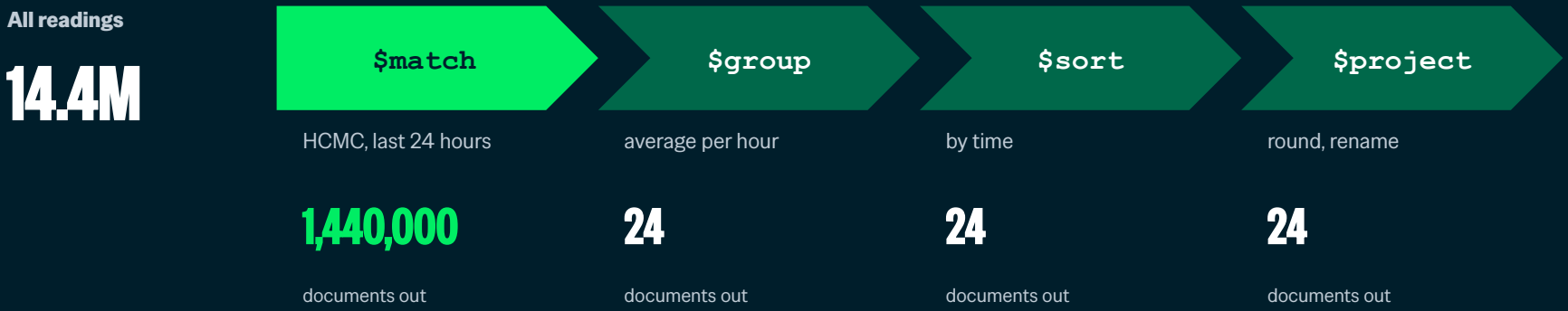
Limitations

Feature	Time series	What to do instead
Unique indexes	Not supported	Remove duplicates before insert, or $group when reading
Writes in multi-document transactions	Not supported	Keep transactional data in a normal collection
Change streams, Atlas Triggers, Atlas Search	Not supported	Stream from your ingest layer (Kafka, app code)
Update	metaField only	Insert a corrected reading instead
Delete	Yes, 7.0+	Any filter works
Max document size	4 MB	Readings are small, rarely an issue
Turn an existing collection into time series	No	Create a new one and copy with $out

03

AGGREGATION PIPELINE

Aggregation Pipeline in One Picture



Put $match first. The less data enters the line, the less every later stage has to do. On time series data, filtering on the metaField and time lets MongoDB skip whole buckets.

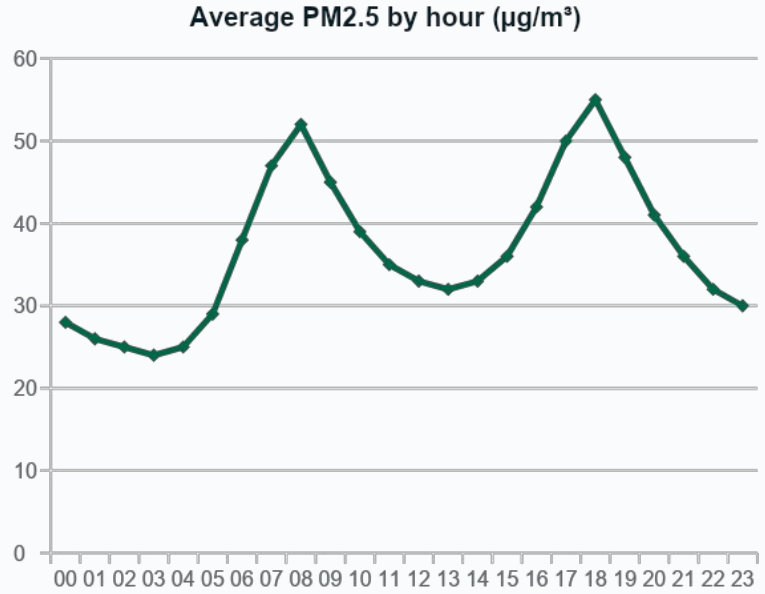
The Stages You'll Use Most

Stage	What it does	In SQL	Example
`$match`	Keep documents that match a filter	WHERE	`{ "sensor.city": "HCMC" }`
`$group`	Combine documents and compute totals	GROUP BY	`{ _id: "$sensor.id", avg: { $avg: "$pm25" } }`
`$sort`	Put results in order	ORDER BY	`{ ts: 1 }  // 1 = oldest first`
`$project`	Choose, rename or compute fields	SELECT	`{ pm25: { $round: ["$pm25", 1] } }`
`$limit`	Keep only the first N	LIMIT	`10`

"$pm25" with a dollar sign means the value of the pm25 field. "pm25" without it is just text.

Hourly Average PM2.5 in HCMC

```
db.air_quality.aggregate([
  { $match: {
      "sensor.city": "HCMC",
      ts: { $gte: ISODate("2026-10-12T00:00:00+07:00"),
            $lt:  ISODate("2026-10-13T00:00:00+07:00") }
  } },
  { $group: {
      // dates are stored in UTC, so pass the timezone
      _id: { $dateTrunc: { date: "$ts", unit: "hour",
             timezone: "Asia/Ho_Chi_Minh" } },
      avgPm25: { $avg: "$pm25" },
      maxPm25: { $max: "$pm25" }
  } },
  { $sort: { _id: 1 } }
])
```



Sample data for illustration.

Time-Aware Operators

$dateTrunc

Round a timestamp down to the minute, hour, day, week or month. 08:47:12 becomes 08:00.

```
{ $dateTrunc: {
    date: "$ts", unit: "day",
    timezone: "Asia/Ho_Chi_Minh" } }
```

$setWindowFields

Calculate across neighbouring readings: moving averages, running totals.

```
avg3h: { $avg: "$avgPm25",
  window: {
    documents: [-2, 0] } }
```

$densify + $fill

A sensor went offline? Create the missing time slots, then fill them in.

```
{ $densify: { field: "ts",
  range: { step: 1, unit: "hour",
    bounds: "full" } } }
```

Query Performance Tips

1. **Filter on metaField and time first**

   MongoDB checks each bucket's min and max and skips the ones that cannot match.

2. **Add an index that matches your filter**

```
db.air_quality.createIndex({  "sensor.id": 1, ts: -1 })
```

3. **Check the plan with explain()**

```
db.air_quality.explain( "executionStats" ).aggregate([...])
```

MongoDB 8.0+

Block processing

Pipelines that start with $match, $sort and $group on time series data can work on whole blocks of values at once instead of one document at a time.

Nothing to configure. MongoDB reports big speed-ups in its own benchmarks; measure on your own data.

04

DATA LIFECYCLE MANAGEMENT

Hot, Warm and Cold Data



HOT	WARM	COLD
Raw readings in the time series collection	Hourly or daily summaries in a normal collection	Older data on low-cost storage
Dashboards, alerts	Trends, monthly reports	Audits, rare lookups
`expireAfterSeconds`	`$merge rollups`	`Online Archive`

Ages are an example. Pick yours from how often each kind of question gets asked.

Expiring Old Data (TTL)

```
// when creating the collection
db.createCollection("air_quality", {
  timeseries: { timeField: "ts", metaField: "sensor" },
  expireAfterSeconds: 7776000  // 90 days
})

// or change it later
db.runCommand({ collMod: "air_quality",
  expireAfterSeconds: 2592000 }) // 30 days
```

Different retention per sensor type: use a partial TTL index filtered on the metaField (6.3+).

1. **Runs every 60 seconds**

   A background job looks for expired data.

2. **Deletes whole buckets**

   Only once every reading in the bucket has expired.

3. **Can run a little late**

   Up to one bucket span, about 24 hours with "minutes". Plan for it if exact deletion dates matter.

Downsampling With $merge

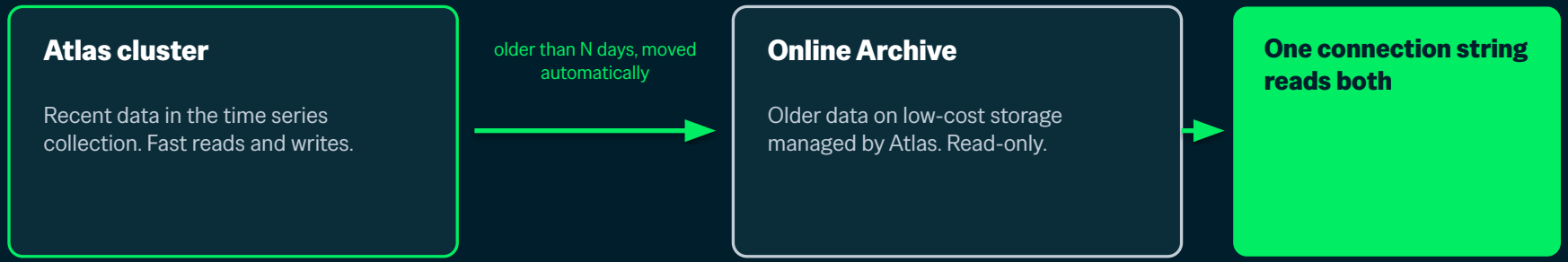
```
db.air_quality.aggregate([
  { $match: { ts: {
      $gte: ISODate("2026-10-12T00:00:00+07:00"),
      $lt:  ISODate("2026-10-13T00:00:00+07:00") } } },
  { $group: {
      _id: { sensor: "$sensor.id",
             day: { $dateTrunc: { date: "$ts", unit: "day",
                    timezone:"Asia/Ho_Chi_Minh" } } },
      avgPm25: { $avg: "$pm25" },
      maxPm25: { $max: "$pm25" },
      readings: { $count: {} } } },
  { $merge: { into: "air_quality_daily",
              whenMatched:"replace" } }
])
```

1,440 → 1

documents per sensor per day

- Run it once a day with an Atlas Scheduled Trigger or a cron job.
- $merge writes into a normal collection. To write into a time series collection, use $out.
- Keep raw data for weeks, summaries for years.

Atlas Online Archive



- The archive rule is based on the collection's timeField.
- Needs an M10 or larger cluster, so it is not on the free M0 tier.
- If you also use TTL, set it longer than the archive age, or data gets deleted before it is archived.

Hackathon Cheat Sheet

1. Readings over time go in a time series collection. Records you edit go in a normal one.
2. timeField holds a real Date, never a string
3. metaField is the source ID plus fixed labels you filter on
4. granularity follows how often ONE source sends data
5. Write with insertMany and { ordered: false }
6. Start pipelines with $match on metaField and time
7. Add timezone: "Asia/Ho_Chi_Minh" to date operators
8. Plan retention: TTL, daily rollups, archive

Project ideas: air quality dashboard, delivery fleet tracker, stock candlestick chart, smart energy meter

Resources

Topic	Link
Time series overview	mongodb.com/docs/manual/core/timeseries-collections
Best practices	mongodb.com/docs/manual/core/timeseries/timeseries-best-practices
Limitations	mongodb.com/docs/manual/core/timeseries/timeseries-limitations
Aggregation pipeline	mongodb.com/docs/manual/core/aggregation-pipeline
Atlas Online Archive	mongodb.com/docs/atlas/online-archive/overview
Free Atlas cluster	mongodb.com/cloud/atlas/register
MongoDB University	learn.mongodb.com

06

Q&A